

APPENDIX 3 / RUST BACKEND MASTERY

付録 — 逆引き: やりたいことからコードを探す

「ファイルを読みたい」「幅をそろえて表示したい」から、書き方を 732 個引く。本文に出てくる所と、合わせて覚えるコードつき

付録: 逆引き — やりたいことからコードを探す

この逆引きについて

付録 2「出てくるコードとコマンドの一覧」は、「この `.collect()` は何だっけ？」のように、**コードから意味を引く**ための物です。この付録 3 は、その反対です。「**ファイルを読みたい**」「**表の形に幅をそろえて表示したい**」のような、**やりたいことから、書き方を引く**ための物です。

- **全部で 732 個**の「やりたいこと」を、**32 の分類**に分けています。下の「分類の一覧」から分類を選ぶか、HTML 版（`Rust\html`）で `Ctrl` ＋ `F` を押して、やりたいことの言葉（「読む」「並べかえる」「消す」など）で探してください。
- **「書き方」の列:** そのまま使える形の、短いコードです。`…` の所は、自分の場面に合わせて書きます。`／` で区切って、いくつかの書き方を並べた行もあります。
- **「ひとこと」の列:** 気をつけることや、結果の例です。くわしい意味は、付録 2 の一覧と、本文の「▶ ここで初めて出てくるコード」の箱にあります。
- **「出てくる所」の列:** その書き方が、この本で初めて出てくる巻と UNIT です（656 個）。実際にどう使われているかを、そこで見られます。
- **「＋ 合わせて覚える」:** 本文で使った書き方と一緒に覚えておくと、すぐ役に立つ書き方です（76 個）。本文のコードとは別に、この付録で足しました。使う前に、付録 2 や Rust の説明書（`rustup doc`）で、くわしい使い方を確かめてください。

> **ヒント:** 1 つの分類の表を、上から下まで一度ながめておくと、「こういうこともできるんだ」と分かり、あとで探す時に思い出しやすくなります。特に「文字列」「Vec と配列」「イテレーター」「エラー処理」は、どのプログラムでもよく使います。

分類の一覧

分類	数	分類	数
Rust の道具を入れる・確かめる	15	所有権・借用・スマートポインタ	15
プロジェクトを作る・動かす（cargo）	44	イテレーター（1 つずつ取り出して処理する）	28
PowerShell の基本の操作	62	ファイルとフォルダ	29
Git（変更の記録）	23	入力・引数・環境変数・ほかのプログラム	12

分類	数	分類	数
Docker と Fly.io	23	時間	10
WSL と bash（Linux のコマンド）	18	スレッドと async（同時に動かす）	17
画面に出す	33	Web の画面と URL（Topcoat）	26
変数と数	29	データベースと SQL	41
文字列	32	JSON（serde）	13
Vec と配列（並び）	32	ログ（tracing）	9
HashMap と HashSet（表と集まり）	14	テスト	14
条件・くり返し・場合分け	29	安全（パスワード・合言葉・HTML）	10
Option（あるかもしれない値）	16	HTML と CSS	27
エラー処理（Result）	19	正規表現（文字の形で探す）	14
関数・構造体・enum・トレイト	27	設定ファイル（Cargo.toml・fly.toml・Dockerfile・GitHub Actions）	25
モジュールと公開	16	ブラウザーで確かめる（Playwright・JavaScript）	10

Rust の道具を入れる・確かめる

Rust そのもの（`rustc`・`cargo`）と、その部品の入れ方・版の確かめ方です。

やりたいこと	書き方	ひとこと	出てくる所
Rust が入っているか、版を確かめる	`rustc --version` `cargo --version`	版が表示されれば入っている	第0巻 はじめに: 手を動かす準備
くわしい版（どのパソコン向けか）を見る	`rustc -vV`	host: の行が、このパソコンの種類	第1巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
入っている版と、今使う版を見る	`rustup show`	版がいくつも入っている時に便利	第1巻 UNIT 01
Rust を新しい版にする	`rustup update`	安定版（stable）を最新にする	第1巻 UNIT 04
決まった版を入れる	`rustup toolchain install 1.99.1 --profile minimal`	最小限の部品だけ入れる	第1巻 UNIT 04
入っている版の一覧を見る	`rustup toolchain list`	今使う版に (default) が付く	第1巻 UNIT 04
いつも使う版を切りかえる	`rustup default 1.99.1`	パソコン全体の既定を変える	＋ 合わせて覚える
このフォルダだけ版を決める（rust-toolchain.toml）	`[toolchain]` `channel = "1.99.1"`	フォルダに置くと自動で切りかわる	第1巻 UNIT 01
1 回だけ別の版で動かす	`cargo +1.99.0 build`	入れてある版に限る	第1巻 UNIT 06
部品（rustfmt・clippy）を足す	`rustup component add rustfmt clippy`	コードを整える係と、気になる所を教える係	第1巻 UNIT 04
入っている部品を見る	`rustup component list --installed`		第1巻 UNIT 04
ほかのパソコン向けの部品を足す	`rustup target add x86_64-pc-windows-msvc`	cargo build --target と組で使う	第1巻 UNIT 05
エラーの番号のくわしい説明を読む	`rustc --explain E0384`	エラーの文の [E0384] の部分	第0巻 UNIT 09
Rust の説明書を手元で開く	`rustup doc`	標準ライブラリの説明が、ネットが無くても読める	＋ 合わせて覚える
Rust を消す	`rustup self uninstall`	rustup で入れた物を全部消す	第1巻 UNIT 04

プロジェクトを作る・動かす（cargo）

Cargo は、Rust のプロジェクトの作成・翻訳・実行・テスト・部品の管理をまとめて行う道具です。`Cargo.toml` があるフォルダで打ちます。

やりたいこと	書き方	ひとこと	出てくる所
新しいプロジェクトを作る	`cargo new practice`	`practice\Cargo.toml` と `src\main.rs` ができる	第0巻 はじめに: 手を動かす準備
部品（ライブラリ）のプロジェクトを作る	`cargo new --lib quality`	`src\lib.rs` ができる	第1巻 UNIT 06
今いるフォルダをプロジェクトにする	`cargo init`	中身のあるフォルダで使う	＋ 合わせて覚える
翻訳して動かす	`cargo run`	変わっていなければ翻訳を省く	第0巻 はじめに: 手を動かす準備
Cargo の表示を出さずに動かす	`cargo run -q`	プログラムの出力だけ見る	第0巻 UNIT 03
プログラムに引数を渡す	`cargo run -- Rust "Web アプリ"`	-- の後ろがプログラムへ	第0巻 UNIT 03
速い本番用で動かす	`cargo run --release`	速さを測る時は必ずこれ	第0巻 UNIT 04
`src\bin` の中の 1 つを動かす	`cargo run --bin u01_stack_heap`	ファイル名から .rs を除いた名前	第3巻 UNIT 01
examples の中の 1 つを動かす	`cargo run --example u01_idor`		第8巻 UNIT 01
ワークスペースの 1 つを動かす	`cargo run -p app`	-p はパッケージの名前	第4巻 UNIT 06
機能（feature）を ON にして動かす	`cargo run --features json`		第4巻 UNIT 07
既定の機能を OFF にして動かす	`cargo run --no-default-features`		第4巻 UNIT 07
翻訳だけする	`cargo build`	`target\debug` にできる	第0巻 UNIT 01
本番用に翻訳する	`cargo build --release`	`target\release` にできる	第11巻 UNIT 09
Cargo.lock のまま翻訳する	`cargo build --locked`	版がずれていたら止まる	第4巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
ほかのパソコン向けに翻訳する	`cargo build --target x86_64-pc-windows-msvc`		第1巻 UNIT 05
エラーが無いかだけ速く調べる	`cargo check`	実行ファイルを作らない	第1巻 UNIT 06
コードを整える	`cargo fmt`	字下げ・空白をそろえる	第1巻 UNIT 06
整っているかだけ調べる	`cargo fmt --check`	CI で使う	第1巻 UNIT 06
気になる所を教えてもらう	`cargo clippy`	直し方も教えてくれる	第0巻 UNIT 09
警告をエラーとして扱う	`cargo clippy --all-targets -- -D warnings`	警告ゼロを守らせる	第1巻 UNIT 03
部品を足す	`cargo add serde --features derive`	Cargo.toml に書き足される	第4巻 この巻で身に付けること
テストの時だけの部品を足す	`cargo add --dev tempfile`	[dev-dependencies] に入る	第11巻 UNIT 03
部品を外す	`cargo remove serde`	Cargo.toml から消える	第14巻 A. 道具の早見表
部品を新しい版にする	`cargo update`	Cargo.lock を書きかえる	第9巻 UNIT 05
1 つの部品だけ新しくする	`cargo update -p tokio`		第9巻 UNIT 05
新しくすると何が変わるか見る	`cargo update --dry-run`	本当には変えない	第9巻 UNIT 05
使っている部品の木を見る	`cargo tree --depth 1`	直接使っている物だけ	第9巻 UNIT 05
その部品をだれが使っているか調べる	`cargo tree -i rustls-webpki`	古い版の出どころ探し	第11巻 UNIT 10
crates.io で部品を探す	`cargo search topcoat --limit 1`	最新の版も分かる	第9巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
説明書（ドキュメント）を作って開く	`cargo doc --open`	ブラウザーで開く	＋ 合わせて覚える
自分のクレートだけ説明書を作る	`cargo doc --no-deps`		第4巻 UNIT 10
翻訳した物を消す	`cargo clean`	target フォルダを消す。容量を空けたい時に	第12巻 UNIT 01
道具（コマンド）を入れる	`cargo install topcoat-cli --version 0.11.0 --locked`		第1巻 UNIT 08
入れた道具の一覧を見る	`cargo install --list`		＋ 合わせて覚える
テストを全部動かす	`cargo test`	#[test] の付いた関数を探して動かす	第0巻 UNIT 10
名前の一部でテストをしぼる	`cargo test title`	名前に title を含む物だけ	第0巻 UNIT 10
テストの中の println! を見る	`cargo test -- --nocapture`		第8巻 UNIT 01
テストを 1 つずつ順に動かす	`cargo test -- --test-threads=1`	ファイルやポートがぶつかる時	第8巻 UNIT 01
tests フォルダの 1 ファイルだけ動かす	`cargo test --test http`		第8巻 UNIT 01
説明の中の例だけテストする	`cargo test --doc`		第4巻 UNIT 10
飛ばすテスト（#[ignore]）を動かす	`cargo test -- --ignored`		第8巻 UNIT 01
テストの名前の一覧を見る	`cargo test -- --list`		第8巻 UNIT 01
速さを測る	`cargo bench`	benches フォルダのコード	第11巻 UNIT 08

PowerShell の基本の操作

Windows で、コマンドを打つ画面の使い方です。`#` から後ろはコメントです。

やりたいこと	書き方	ひとこと	出てくる所
今いるフォルダを知る	`pwd` `Get-Location`	行の頭の PS `C:\…>` にも出ている	第1巻 UNIT 02
フォルダを移る	`cd rust-study`	cd .. で 1 つ上、cd ~ でホームへ	第0巻 はじめに: 手を動かす準備
フォルダの中を見る	`ls` `Get-ChildItem`	dir でも同じ	第0巻 UNIT 03
中のフォルダまで全部見る	`Get-ChildItem -Recurse -Filter *.rs`	.rs のファイルを全部探す	第14巻 A. 道具の早見表
隠れたファイルも見る	`Get-ChildItem -Force`	.git などが見える	＋ 合わせて覚える
フォルダを作る	`mkdir rust-study` `New-Item -ItemType Directory -Force logs`	-Force はもうあってもエラーにしない	第0巻 はじめに: 手を動かす準備
ファイルを作る・中身を書く	`Set-Content -Path memo.txt -Value "こんにちは" -Encoding utf8`	前の中身は消える	第0巻 UNIT 03
ファイルに書き足す	`Add-Content -Path log.txt -Value "1 行"`	後ろに足す	＋ 合わせて覚える
ファイルの中身を見る	`Get-Content memo.txt` `cat memo.txt`		第0巻 UNIT 01
ファイルの最後の数行を見る	`Get-Content app.log -Tail 20`	-Wait を付けると増えるのを待ち続ける	＋ 合わせて覚える
ファイルをコピーする	`Copy-Item a.txt b.txt`	フォルダごとは -Recurse	第0巻 UNIT 03
ファイルを動かす・名前を変える	`Move-Item a.txt old\a.txt` `Rename-Item a.txt b.txt`		第14巻 A. 道具の早見表
ファイルを消す	`Remove-Item memo.txt`	ごみ箱に入らないので注意	第0巻 UNIT 03
フォルダを中身ごと消す	`Remove-Item -Recurse -Force target`	場所をよく確かめる	第14巻 A. 道具の早見表
ファイルやフォルダがあるか調べる	`Test-Path .env`	True か False	第1巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
コマンドがどこにあるか調べる	`Get-Command cargo` `(Get-Command cargo).Source`	どの cargo が動くか	第1巻 UNIT 05
ファイルの中から文字を探す	`Select-String -Path *.rs -Pattern "unwrap"`	grep の代わり	第14巻 A. 道具の早見表
画面を消す	`cls` `Clear-Host`		＋ 合わせて覚える
前に打ったコマンドを出す	`↑ キー` `Get-History`	Ctrl＋R で探せる	＋ 合わせて覚える
動いているコマンドを止める	`Ctrl ＋ C`	cargo run のサーバーを止める時も	第0巻 UNIT 06
変数に入れる	`$name = "Rust"`	$ で始める	第1巻 UNIT 01
環境変数を入れる（この窓だけ）	`$env:RUST_LOG = "debug"`	窓を閉じると消える	第0巻 UNIT 03
環境変数を消す	`Remove-Item Env:RUST_LOG`		第0巻 UNIT 03
ずっと残る環境変数を入れる	`[Environment]::SetEnvironmentVariable("名前", "値", "User")`	新しい窓から効く	＋ 合わせて覚える
最後のプログラムが成功したか見る	`$LASTEXITCODE`	0 なら成功	第0巻 UNIT 01
出力をファイルに書く	`cargo run > out.txt`	>> なら書き足す	第0巻 UNIT 01
エラーの出力も一緒に書く	`cargo build 2>&1 | Tee-Object build.log`	画面とファイルの両方へ	第0巻 UNIT 01
エラーの出力を捨てる	`git --version 2>$null`		第1巻 UNIT 10
結果を次のコマンドに渡す	`Get-Service | Where-Object Status -eq Running`	| はパイプ	第0巻 UNIT 06
条件でしぼる	`Where-Object { $_.Length -gt 1MB }`	$_ は流れてきた 1 つ	第1巻 UNIT 09
1 つずつ処理する	`ForEach-Object { $_.Name }`		第1巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
並べかえる	`Sort-Object Length -Descending`	大きい順	＋ 合わせて覚える
列を選ぶ・最初の数件だけ	`Select-Object Name, Length -First 5`		第0巻 UNIT 06
数える・合計する	`Measure-Object -Sum Length`	.Count と .Sum	＋ 合わせて覚える
表の形で見る	`Format-Table -AutoSize`	列が切れる時	第1巻 UNIT 09
幅をそろえて文字を作る	`"{0,-10} {1}" -f $name, $size`	-10 は 10 文字分の左寄せ	第1巻 UNIT 04
数を 3 けた区切りにする	`"{0:N0}" -f 1234567`	1,234,567	＋ 合わせて覚える
日時を文字にする	`Get-Date -Format "yyyyMMdd_HHmm"`	MM は月、mm は分	第12巻 UNIT 08
文字の中に変数を入れる	`"こんにちは、$name さん"`	"..." の中だけ	第1巻 UNIT 05
文字の中に式の結果を入れる	`"大きさ: $($file.Length) バイト"`	$( ) で囲む	第1巻 UNIT 10
そのままの文字にする	`'これは $name のまま'`	'...' の中は置きかわらない	第1巻 UNIT 01
何行もの文字を書く	`@' ～ '@`	'@ は行の頭に書く	第1巻 UNIT 01
長いコマンドを次の行へ続ける（行の最後に書く）	`` ` ``	バッククォート（Shift＋@）の後ろに空白を入れない	第1巻 UNIT 05
空白の入った場所のプログラムを動かす	`& "C:\Program Files\Git\bin\git.exe" --version`	先頭の &	第1巻 UNIT 05
今いるフォルダのプログラムを動かす	`.\target\debug\practice.exe`	`.\` が要る	第0巻 UNIT 01
スクリプトを動かす	`powershell -ExecutionPolicy Bypass -File .\doctor.ps1`	この 1 回だけ制限を外す	第1巻 UNIT 10

やりたいこと	書き方	ひとこと	出てくる所
エラーが起きたら止める	`$ErrorActionPreference = "Stop"`	スクリプトの最初に	第12巻 UNIT 08
エラーを受け止める	`try { … } catch { $_.Exception.Message }`		第1巻 UNIT 10
並び・表を作る	`@("a", "b")` `@{ Name = "web"; Port = 8080 }`		第1巻 UNIT 02
くり返す	`foreach ($f in Get-ChildItem) { $f.Name }` `1..3 | ForEach-Object { $_ }`		第1巻 UNIT 05
ポートを使っているプログラムを探す	`Get-NetTCPConnection -LocalPort 8080 -State Listen`	.OwningProcess が番号	第0巻 UNIT 06
番号でプログラムを止める	`Stop-Process -Id 1234`		＋ 合わせて覚える
名前でプログラムを探す	`Get-Process cargo`		第0巻 UNIT 06
Web のページを取ってくる	`Invoke-WebRequest http://127.0.0.1:3000/ -UseBasicParsing`	.StatusCode・.Content	＋ 合わせて覚える
curl で要求を送る	`curl.exe -s -i http://127.0.0.1:3000/`	.exe を付ける（PowerShell の curl は別物）	第0巻 UNIT 06
POST で送る	`curl.exe -i -d "title=牛乳" http://127.0.0.1:3000/notes`		第1巻 UNIT 09
ヘッダーを付けて送る	`curl.exe -H "Content-Type: application/json" --data-binary "@note.json" URL`		第6巻 UNIT 06
状態コードだけ見る	`curl.exe -s -o NUL -w '%{http_code}' URL`		第1巻 UNIT 09
ファイルの指紋（ハッシュ）を出す	`Get-FileHash file.zip -Algorithm SHA256`	同じ物かの確かめ	第1巻 UNIT 04
ZIP にまとめる・ほどく	`Compress-Archive -Path dist\* -DestinationPath app.zip` `Expand-Archive app.zip out`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
かかった時間を測る	`Measure-Command { cargo build }`	.TotalSeconds	＋ 合わせて覚える
PowerShell の版を見る	`$PSVersionTable.PSVersion`	5.1 か 7.x	第1巻 UNIT 01

Git（変更の記録）

ファイルの変更を記録して、いつでも戻れるようにする道具です。

やりたいこと	書き方	ひとこと	出てくる所
Git の版を見る	`git --version`		第1巻 UNIT 02
名前とメールを設定する（最初に 1 回）	`git config --global user.name "名前"`	user.email も	第1巻 UNIT 02
設定を一覧で見る	`git config --list --show-origin`	どのファイルの設定か分かる	第1巻 UNIT 02
フォルダを Git の管理にする	`git init`	.git フォルダができる	第1巻 UNIT 02
今の状態を見る	`git status` `git status --short`	変わったファイルの一覧	第1巻 UNIT 02
記録に入れるファイルを選ぶ	`git add src/main.rs` `git add .`	. は全部	第1巻 UNIT 02
記録する（コミット）	`git commit -m "説明"`		第1巻 UNIT 02
記録の一覧を見る	`git log --oneline`	1 件 1 行	第1巻 UNIT 02
何を変えたか見る	`git diff` `git diff --staged`	add した後は --staged	第1巻 UNIT 03
変更を取り消して元に戻す	`git restore src/main.rs`	保存前の変更は消える	第1巻 UNIT 03
add を取り消す	`git restore --staged src/main.rs`	ファイルの中身は残る	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
ファイルを管理から外す（消さない）	`git rm --cached .env`	秘密を入れてしまった時	第1巻 UNIT 02
無視するファイルを決める（.gitignore に書く）	`target/` `.env`	1 行に 1 つ	＋ 合わせて覚える
なぜ無視されるか調べる	`git check-ignore -v .env`		第1巻 UNIT 02
改行の種類を調べる	`git ls-files --eol`	LF か CRLF か	第1巻 UNIT 02
ある記録の中身を見る	`git show HEAD`		第1巻 UNIT 02
枝（ブランチ）を作って移る	`git switch -c feature`	試しの変更を分ける	＋ 合わせて覚える
枝の一覧を見る	`git branch`		＋ 合わせて覚える
枝を取り込む	`git merge feature`		＋ 合わせて覚える
GitHub に送る	`git push`	最初は git push -u origin main	＋ 合わせて覚える
GitHub から受け取る	`git pull`		＋ 合わせて覚える
リポジトリを丸ごと写す	`git clone https://github.com/…`		＋ 合わせて覚える
変更を一時的にしまう	`git stash` `git stash pop`		＋ 合わせて覚える

Docker と Fly.io

アプリを「コンテナ」に入れて、どこでも同じように動かす道具（Docker）と、それをインターネットに公開する所（Fly.io）です。

やりたいこと	書き方	ひとこと	出てくる所
Docker が動くか確かめる	`docker version`	Server の欄が出れば動いている	第1巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
イメージを作る	`docker build -t rustnote:v1 .`	. は Dockerfile のあるフォルダ	第11巻 UNIT 10
作る途中を全部表示する	`docker build --progress=plain -t rustnote .`		第12巻 UNIT 02
イメージの一覧を見る	`docker image ls`		第1巻 UNIT 07
コンテナを動かして、止まったら消す	`docker run --rm rustnote:v1`		第1巻 UNIT 07
裏で動かして、ポートをつなぐ	`docker run -d --name app -p 8080:8080 rustnote:v1`	左がパソコン、右がコンテナ	第1巻 UNIT 07
環境変数を渡す	`docker run -e DATABASE_URL=… rustnote:v1`		第12巻 UNIT 05
フォルダをつなぐ	`docker run -v ${PWD}/data:/data rustnote:v1`	コンテナを消しても data は残る	第12巻 UNIT 10
動いているコンテナを見る	`docker ps` `docker ps -a`	-a は止まった物も	第1巻 UNIT 09
コンテナのログを見る	`docker logs app` `docker logs -f app`	-f は増えるのを待ち続ける	＋ 合わせて覚える
コンテナを止める・消す	`docker stop app` `docker rm app`		第12巻 UNIT 10
コンテナの中でコマンドを打つ	`docker exec -it app sh`	中を調べる時に	＋ 合わせて覚える
イメージの層の大きさを見る	`docker history rustnote:v1`		第12巻 UNIT 02
イメージに名前を付け直す	`docker tag rustnote:v1 registry.fly.io/app:v1`		第12巻 UNIT 10
コンテナどうしをつなぐ網を作る	`docker network create rn-net`		第12巻 UNIT 06
Fly.io の道具の版を見る	`fly version`		第1巻 UNIT 10
アプリを登録する（公開はまだ）	`fly launch --no-deploy --name 自分のアプリ名`	fly.toml ができる	第12巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
公開する	`fly deploy`		第12巻 UNIT 08
秘密の値を渡す	`fly secrets set DATABASE_AUTH_TOKEN=…`	fly.toml には書かない	第12巻 UNIT 05
状態を見る	`fly status`		第12巻 UNIT 08
ログを見る	`fly logs`		第12巻 UNIT 08
公開した版の一覧を見る	`fly releases --image`	前の版に戻す時に	第12巻 UNIT 10
前の版に戻す	`fly deploy --image registry.fly.io/…:前の札`		第12巻 UNIT 10

WSL と bash（Linux のコマンド）

Windows の中で Linux を動かす WSL と、Linux のコマンドを打つ bash の使い方です。

やりたいこと	書き方	ひとこと	出てくる所
WSL を入れる	`wsl --install`	管理者の PowerShell で。終わったら再起動	第1巻 UNIT 07
入っている Linux を見る	`wsl -l -v`	WSL の版（2）と状態	第1巻 UNIT 07
Linux でコマンドを 1 つ動かす	`wsl -d Ubuntu -- uname -m`		第1巻 UNIT 07
今いるフォルダを知る	`pwd`		第1巻 UNIT 02
フォルダを作る（途中もまとめて）	`mkdir -p a/b/c`		第1巻 UNIT 07
中身ごと消す	`rm -rf フォルダ`	戻せないので場所をよく確かめる	第1巻 UNIT 07
ファイルの中を見る	`cat file` `less file`	less は q で終わる	第0巻 UNIT 03

やりたいこと	書き方	ひとこと	出てくる所
見えない文字も見る	`cat -v crlf.sh`	CR が ^M に見える	第1巻 UNIT 02
CRLF を LF に直す	`sed -i 's/\r$//' crlf.sh`		第1巻 UNIT 02
ファイルの中から探す	`grep -n "unwrap" src/*.rs`	-n は行番号	＋ 合わせて覚える
権限を変える	`chmod +x run.sh`	+x で実行できるようにする	＋ 合わせて覚える
持ち主を変える	`chown app:app /data`		第12巻 UNIT 02
部品（パッケージ）を入れる	`sudo apt-get update && sudo apt-get install -y build-essential`		第12巻 UNIT 02
失敗したら止めるスクリプトにする	`set -e`	スクリプトの最初に	第1巻 UNIT 02
前が成功したら次を動かす	`cargo build && cargo test`		第1巻 UNIT 07
前が失敗したら次を動かす	`cd dir || exit 1`		第1巻 UNIT 07
コマンドの結果を使う	`echo "今日は $(date +%F)"`		第1巻 UNIT 07
URL のスクリプトを取ってきて動かす	`curl -sSfL https://… | bash`	信用できる所の物だけ	第1巻 UNIT 09

画面に出す

`println!` などの「文字の中の書式」（`{ }` の中の `:` の後ろ）で、幅・寄せ方・けたを決められます。`format!` で同じ書き方の文字を作れます。

やりたいこと	書き方	ひとこと	出てくる所
1 行出す（最後に改行）	`println!("こんにちは");`		第0巻 はじめに: 手を動かす準備

やりたいこと	書き方	ひとこと	出てくる所
改行せずに出す	`print!("入力: ");`	すぐ出したい時は io::stdout().flush()	第0巻 UNIT 02
エラーの文として出す	`eprintln!("失敗: {e}");`	画面には出るが、> out.txt には入らない	第0巻 UNIT 01
値を入れて出す	`println!("{} 点", score);`	{} に後ろの値が入る	第0巻 UNIT 01
変数の名前を中に書いて出す	`println!("{name} さん");`	{} の中は名前だけ	第0巻 UNIT 01
同じ値を番号で何度も使う	`println!("{0} と {1} と {0}", a, b);`		第4巻 UNIT 05
プログラマー向けの形で出す	`println!("{:?}", v);`	Vec や Option も出せる	第0巻 UNIT 02
項目ごとに改行して見やすく出す	`println!("{:#?}", note);`		第2巻 UNIT 07
幅をそろえて左寄せで出す	`println!("{name:<22}|");`	22 文字分。足りない分は右に空白	第0巻 UNIT 02
右寄せで出す	`println!("{:>8}", price);`	数の表のけたがそろう	第3巻 UNIT 03
真ん中に出す	`println!("{:^20}", "見出し");`		第14巻 B. サンプル集
空白でなく決めた字で埋める	`println!("{:-<20}", "");`	- を 20 個（区切り線）	＋ 合わせて覚える
幅を変数で決める	`println!("{:>w$}", n, w = 6);`	w$ が「幅は w」	＋ 合わせて覚える
前を 0 で埋めて、けたをそろえる	`println!("{:02}:{:02}", h, m);`	9:5 → 09:05	第0巻 UNIT 08
小数点の下を何けた出すか決める	`println!("{:.1}", 3.14159);`	3.1	第2巻 UNIT 02
幅と小数のけたを両方決める	`println!("{:8.2}", x);`	8 文字分の幅に、小数 2 けた	＋ 合わせて覚える
+ の印も付ける	`println!("{:+}", 5);`	+5	＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
16 進数で出す	`println!("{:x} {:X} {:#x}", 255, 255, 255);`	ff FF 0xff	第0巻 UNIT 08
2 進数で出す	`println!("{:b}", 10);`	1010	＋ 合わせて覚える
8 進数で出す	`println!("{:o}", 8);`	10	＋ 合わせて覚える
指数の形で出す	`println!("{:e}", 1234.5);`	1.2345e3	＋ 合わせて覚える
メモリの場所を出す	`println!("{:p}", &x);`		第3巻 UNIT 02
{ と } そのものを出す	`println!("{{ }}");`	2 つ重ねる	第5巻 UNIT 09
エラーを原因までつないで出す	`println!("{:#}", err);`	anyhow のエラーで	第14巻 B. サンプル集
画面に出さずに文字を作る	`let s = format!("{} 件", n);`	String ができる	第0巻 UNIT 06
文字に書き足す	`write!(s, "{n}")?;`	use std::fmt::Write; が要る	第2巻 UNIT 09
ファイルに 1 行書く	`writeln!(file, "{}", text)?;`	use std::io::Write; が要る	第0巻 UNIT 07
調べ物で、式と値を一緒に出す	`dbg!(&v);`	[src/main.rs:5] &v = [1, 2] のように出る	＋ 合わせて覚える
空の行を出す	`println!();`		第0巻 はじめに: 手を動かす準備
自分の型を {} で出せるようにする	`impl fmt::Display for Note { fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result { write!(f, "{}", self.title) } }`		第2巻 UNIT 09
文字列の中で改行する	`"1行目\n2行目"`		第0巻 UNIT 02
タブで区切る	`"名前\t点数"`		第0巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
" や `\` そのものを入れる	`"\"Rust\""` `"C:\\Users"`	`\` を前に付ける	第4巻 UNIT 04

変数と数

値に名前を付けて覚える方法と、整数・小数の扱い方です。

やりたいこと	書き方	ひとこと	出てくる所
値に名前を付ける	`let age = 20;`	最初は変えられない	第0巻 UNIT 01
あとで変えられる変数にする	`let mut count = 0;`		第0巻 UNIT 01
型を書いて決める	`let total: u64 = 0;`		第0巻 UNIT 04
決まった値（定数）を作る	`const MAX_TITLE: usize = 80;`	名前は大文字	第2巻 UNIT 01
同じ名前で作り直す	`let s = s.trim();`	前の値は使えなくなる（シャドーイング）	第0巻 UNIT 07
使わない値を捨てる	`let _ = fs::remove_file("tmp.txt");`		第0巻 UNIT 07
整数（マイナスあり）	`i32` `i64`	型を書かない整数は i32	第0巻 UNIT 04

整数（0 以上）	`u8`
	`u32`
	`u64`
	`usize`
		長さや何番目かは usize	第0巻 UNIT 10

小数	`f64`		第0巻 UNIT 09
はい・いいえ	`bool`	true か false	第0巻 UNIT 10
1 文字	`char`	'あ' のように ' で囲む	第2巻 UNIT 02

やりたいこと	書き方	ひとこと	出てくる所

足す・引く・掛ける・割る	`a + b`
	`a - b`
	`a * b`
	`a / b`
		整数の割り算は小数を切り捨てる	第0巻 はじめに: 手を動かす準備

割った余り	`a % b`	偶数は n % 2 == 0	第0巻 UNIT 04
足して入れる	`total += n;`	-= *= /= もある	第0巻 UNIT 01
型を変える	`n as f64` `x as u8`	大きい型から小さい型は、はみ出すと切れる	第2巻 UNIT 02
読みやすい数の書き方	`1_000_000`	_ は区切り	第0巻 UNIT 04
型を付けた数	`10u8` `0u64`		第0巻 UNIT 04
16 進数・2 進数で書く	`0xff` `0b1010`		第2巻 UNIT 02
とても大きい・小さい小数	`1e6` `2.5e-3`		第2巻 UNIT 02
文字を数にする	`let n: i32 = "42".parse()?;`	数でない時はエラー	第0巻 UNIT 04
あふれたら None にする	`a.checked_add(b)`	Option で返る	第2巻 UNIT 02
あふれたら上限で止める	`a.saturating_sub(b)`	u32 で 0 より下にならない	第7巻 UNIT 10
大きい方・小さい方	`a.max(b)` `a.min(b)`		第2巻 UNIT 02
範囲に収める	`x.clamp(0, 100)`		第11巻 UNIT 07
絶対値	`x.abs()`		第2巻 UNIT 02
べき乗	`2_i32.pow(10)` `x.powi(2)`	整数は pow、小数は powi	第5巻 UNIT 08

やりたいこと	書き方	ひとこと	出てくる所

四捨五入・切り捨て・切り上げ	`x.round()`
	`x.floor()`
	`x.ceil()`
			第8巻 UNIT 08

平方根	`x.sqrt()`		＋ 合わせて覚える
型の一番大きい値	`u32::MAX` `i64::MIN`		第2巻 UNIT 02

文字列

文字列には、借りて見るだけの `&str` と、自分で持って書き足せる `String` の 2 つがあります。

やりたいこと	書き方	ひとこと	出てくる所
文字列を作る	`let s = String::from("Rust");` `"Rust".to_string()`		第0巻 UNIT 03
空の文字列を作る	`let mut s = String::new();`		第0巻 UNIT 05
後ろに書き足す	`s.push_str("の本");` `s.push('!');`	push は 1 文字	第2巻 UNIT 05
つなげた文字を作る	`format!("{a}{b}")` `a + &b`		第0巻 UNIT 06
バイトの長さ	`s.len()`	日本語は 1 文字 3 バイト	第0巻 UNIT 06
文字の数	`s.chars().count()`	見た目の文字数に近い	第0巻 UNIT 05
空か調べる	`s.is_empty()`		第1巻 UNIT 06
前後の空白を取る	`s.trim()`	trim_start・trim_end もある	第0巻 UNIT 07
区切りで分ける	`s.split(',')`	イテレーターが返る	第0巻 UNIT 05
最初の区切りで 2 つに分ける	`s.split_once('=')`	Option<(&str, &str)>	第0巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
空白で分ける	`s.split_whitespace()`	空白がいくつ続いてもよい	第0巻 UNIT 06
1 行ずつ分ける	`text.lines()`	`\r\n` も `\n` も分かる	第0巻 UNIT 07
含むか調べる	`s.contains("rust")`		第0巻 UNIT 10
始まり・終わりを調べる	`s.starts_with("http")` `s.ends_with(".rs")`		第2巻 UNIT 05
頭や後ろの決まった部分を外す	`s.strip_prefix("Bearer ")`	無ければ None	第3巻 UNIT 05
置きかえる	`s.replace("\r\n", "\n")`	新しい String ができる	第2巻 UNIT 05
大文字・小文字にする	`s.to_uppercase()` `s.to_lowercase()`		第2巻 UNIT 05
大文字小文字を気にせず比べる	`a.eq_ignore_ascii_case(b)`	英字だけ	第5巻 UNIT 08
どこにあるか探す	`s.find("@")`	バイトの位置が Option で	第0巻 UNIT 05
一部を取り出す	`&s[0..3]`	バイトの位置。日本語の途中で切ると止まる	第0巻 UNIT 05
先頭から何文字か取る（安全に）	`s.chars().take(10).collect::<String>()`		第0巻 UNIT 05
1 文字ずつ見る	`for c in s.chars() { … }`		第0巻 UNIT 05
同じ文字をくり返す	`"-".repeat(20)`		第0巻 UNIT 10
並びを区切りでつなぐ	`v.join(", ")`		第0巻 UNIT 02
&str から String にする	`s.to_string()` `s.to_owned()`		第0巻 UNIT 07
String から &str にする	`&s` `s.as_str()`		第2巻 UNIT 08

やりたいこと	書き方	ひとこと	出てくる所

文字の種類を調べる	`c.is_ascii_digit()`
	`c.is_alphabetic()`
	`c.is_whitespace()`
			＋ 合わせて覚える

`\` をそのまま書く	`r"C:\Users\you"`	生の文字列	第10巻 UNIT 07
" もそのまま書く	`r#"{"title": "牛乳"}"#`		第4巻 UNIT 08
長い文字を次の行へ続ける	`"長い文字\`	行の終わりの `\`	第0巻 UNIT 05
バイトの並びにする	`s.as_bytes()` `b"abc"`		第0巻 UNIT 05
バイトの並びから文字にする	`String::from_utf8(bytes)?` `String::from_utf8_lossy(&bytes)`	読めない所は `�` に	第6巻 UNIT 07

Vec と配列（並び）

同じ型の値の並びです。数が決まっていれば配列 `[T; N]`、増やしたり減らしたりするなら `Vec<T>` を使います。

やりたいこと	書き方	ひとこと	出てくる所
値を並べて作る	`let v = vec![1, 2, 3];`		第0巻 UNIT 01
空の Vec を作る	`let mut v: Vec<String> = Vec::new();`		第2巻 UNIT 06
大きさを先に取って作る	`Vec::with_capacity(100)`	増やす時の作り直しが減る	第2巻 UNIT 06
同じ値を n 個並べる	`vec![0; 5]` `[0; 5]`		第2巻 UNIT 06
配列を作る	`let a = ["Cargo.toml", "src/main.rs"];`	数は変えられない	第0巻 UNIT 02
後ろに足す	`v.push(4);`		第2巻 UNIT 05
後ろから取り出す	`v.pop()`	空なら None	第2巻 UNIT 06

やりたいこと	書き方	ひとこと	出てくる所
途中に入れる・途中を消す	`v.insert(0, x)` `v.remove(0)`		第0巻 UNIT 09
数を数える	`v.len()`		第0巻 UNIT 06
空か調べる	`v.is_empty()`		第1巻 UNIT 06
何番目かを取り出す	`v[0]`	無い番号だと止まる（パニック）	＋ 合わせて覚える
何番目かを安全に取り出す	`v.get(10)`	無ければ None	第2巻 UNIT 05
最初・最後	`v.first()` `v.last()`	Option で返る	第10巻 UNIT 08
含むか調べる	`v.contains(&3)`		第0巻 UNIT 10
並べかえる	`v.sort();`	小さい順	第2巻 UNIT 06
決めた順に並べかえる	`v.sort_by(|a, b| b.cmp(a));`	大きい順	第14巻 B. サンプル集
項目で並べかえる	`notes.sort_by_key(|n| n.id);`		第2巻 UNIT 10
小数を並べかえる	`v.sort_by(|a, b| a.total_cmp(b));`	f64 は sort が使えない	＋ 合わせて覚える
逆さにする	`v.reverse();`		第14巻 B. サンプル集
隣どうしの重なりを消す	`v.dedup();`	先に sort しておく	第5巻 UNIT 10
条件に合う物だけ残す	`v.retain(|n| !n.done);`		第3巻 UNIT 07
後ろに並びをつなぐ	`v.extend(other);`		第3巻 UNIT 07
全部消す	`v.clear();`		第3巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
決めた長さにする	`v.truncate(10);`		＋ 合わせて覚える
2 つを入れかえる	`v.swap(0, 1);`		＋ 合わせて覚える
一部を借りて見る	`&v[1..3]`	スライス	第0巻 UNIT 08
引数で並びを受け取る	`fn total(xs: &[i32]) -> i32`	Vec も配列も渡せる	第0巻 UNIT 08
n 個ずつに分ける	`v.chunks(3)`	最後は 3 個より少ないことも	第14巻 B. サンプル集
隣り合う組を順に見る	`v.windows(2)`	[a, b], [b, c], …	第5巻 UNIT 08
並んだ物から探す（速く）	`v.binary_search(&x)`	先に sort しておく	＋ 合わせて覚える
文字の並びをつなぐ	`v.join("、")`		第0巻 UNIT 02
大きさの決まった配列の型	`[u8; 32]`		第2巻 UNIT 06

HashMap と HashSet（表と集まり）

キーで値を引ける表（`HashMap`）と、重なりの無い集まり（`HashSet`）です。`use std::collections::HashMap;` が要ります。

やりたいこと	書き方	ひとこと	出てくる所
表を作る	`let mut m = HashMap::new();`		第0巻 UNIT 09
入れる・上書きする	`m.insert("alice", 80);`	前の値は Some で返る	第0巻 UNIT 09
キーで取り出す	`m.get("alice")`	無ければ None	第2巻 UNIT 05
キーで取り出して変える	`if let Some(v) = m.get_mut("alice") { *v += 1; }`		第2巻 UNIT 10

やりたいこと	書き方	ひとこと	出てくる所
無ければ入れて、あれば足す	`*m.entry(word).or_insert(0) += 1;`	数えるのに便利	第2巻 UNIT 10
キーがあるか調べる	`m.contains_key("alice")`		第2巻 UNIT 10
消す	`m.remove("alice")`		第2巻 UNIT 06
全部を順に見る	`for (k, v) in &m { … }`	順番は決まっていない	第0巻 UNIT 01
キーだけ・値だけ	`m.keys()` `m.values()`		第2巻 UNIT 10
キーの順に並んだ表	`BTreeMap::new()`	表示の順をそろえたい時	第2巻 UNIT 10
並びから表を作る	`let m: HashMap<_, _> = pairs.into_iter().collect();`		第0巻 UNIT 05
重なりの無い集まりを作る	`let mut s = HashSet::new();` `s.insert(x)`	insert は初めてなら true	第8巻 UNIT 09
集まりに含むか	`s.contains(&x)`	Vec より速い	第0巻 UNIT 10
並びの重なりを消す	`v.into_iter().collect::<HashSet<_>>()`		＋ 合わせて覚える

条件・くり返し・場合分け

条件で処理を分ける `if`、くり返す `for`・`while`・`loop`、形で分ける `match` です。

やりたいこと	書き方	ひとこと	出てくる所
条件で分ける	`if n > 0 { … } else if n == 0 { … } else { … }`	( ) は要らない	第0巻 UNIT 02
条件で値を選ぶ	`let kind = if is_dir { "フォルダ" } else { "ファイル" };`		第0巻 UNIT 02
等しい・等しくない	`a == b` `a != b`		第0巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所

大きい・小さい	`a < b`
	`a <= b`
	`a >= b`
			第2巻 UNIT 03

かつ・または・ではない	`a && b`
	`a || b`
	`!a`
			第0巻 UNIT 08

範囲の中か	`(1..=9).contains(&n)`		第0巻 UNIT 01
決まった回数くり返す	`for i in 0..5 { … }`	0〜4	第0巻 UNIT 05
終わりの数も含めてくり返す	`for i in 1..=10 { … }`		第0巻 UNIT 01
逆の順にくり返す	`for i in (0..5).rev() { … }`		第2巻 UNIT 04
とびとびにくり返す	`for i in (0..10).step_by(2) { … }`	0, 2, 4, …	第2巻 UNIT 04
並びを順に見る	`for x in &v { … }`	& を付けると v は残る	第0巻 UNIT 01
何番目かも一緒に見る	`for (i, x) in v.iter().enumerate() { … }`	i は 0 から	第0巻 UNIT 03
条件の間くり返す	`while n > 0 { … }`		第2巻 UNIT 01
ずっとくり返す	`loop { … }`	break で抜ける	第0巻 UNIT 06
くり返しを抜ける	`break;`		第0巻 UNIT 06
次の回へ進む	`continue;`		第2巻 UNIT 04
抜ける時に値を返す	`let found = loop { break 42; };`		第2巻 UNIT 04
外側のくり返しまで抜ける	`'outer: for … { break 'outer; }`		第2巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
形で分ける	`match n { 0 => "ゼロ", _ => "ほか" }`		第0巻 UNIT 01
その他すべて	`_ => …`		第0巻 UNIT 06
いくつかの形をまとめる	`1 | 2 | 3 => "小さい",`		第2巻 UNIT 04
範囲で分ける	`0..=59 => "不合格",`		第0巻 UNIT 01
形が合って条件も合う時	`Some(n) if n > 0 => …`		第0巻 UNIT 05
合った値に名前を付ける	`n @ 1..=9 => …`		第2巻 UNIT 08
1 つの形の時だけ処理する	`if let Some(x) = opt { … }`		第0巻 UNIT 05
形である間くり返す	`while let Some(x) = stack.pop() { … }`		第4巻 UNIT 05
形に合うか true/false で知る	`matches!(c, 'a'..='z')`		第2巻 UNIT 08
使わない値を無視する	`(x, _)` `Err(_)`		第2巻 UNIT 04
残りの項目を無視する	`Point { x, .. }`		第2巻 UNIT 08

Option（あるかもしれない値）

値が「ある（`Some`）」か「無い（`None`）」かを表す型です。

やりたいこと	書き方	ひとこと	出てくる所
あるかもしれない値の型	`Option<String>`		第0巻 UNIT 08
値がある・無い	`Some(x)` `None`		第0巻 UNIT 01

やりたいこと	書き方	ひとこと	出てくる所
あるか調べる	`opt.is_some()` `opt.is_none()`		第2巻 UNIT 08
無ければ決めた値にする	`opt.unwrap_or(0)`		第0巻 UNIT 03
無ければ型の初期値にする	`opt.unwrap_or_default()`	文字なら空、数なら 0	第0巻 UNIT 07
無ければ計算して決める	`opt.unwrap_or_else(|| load())`		第4巻 UNIT 05
必ずあると言い切って取り出す	`opt.expect("設定が要ります")`	無いと止まる。理由を書く	第2巻 UNIT 01
中身を変える	`opt.map(|s| s.len())`	None はそのまま	第2巻 UNIT 08
中身から、また Option を作る	`opt.and_then(|s| s.parse().ok())`		第5巻 UNIT 08
条件に合わなければ None にする	`opt.filter(|s| !s.is_empty())`		第2巻 UNIT 10
None ならエラーにする	`opt.ok_or("見つかりません")?`		第2巻 UNIT 09
None ならそこで返す	`let x = opt?;`	Option を返す関数の中で	第0巻 UNIT 02
無ければ早めに抜ける	`let Some(user) = user else { return; };`		第0巻 UNIT 08
借りた形にする	`opt.as_ref()` `opt.as_deref()`	Option → Option<&str> は as_deref	第0巻 UNIT 07
取り出して None にする	`opt.take()`		＋ 合わせて覚える
借りた値を複製した Option にする	`opt.cloned()` `opt.copied()`		第4巻 UNIT 02

エラー処理（Result）

失敗するかもしれない処理は、成功（`Ok`）か失敗（`Err`）かの `Result` を返します。

やりたいこと	書き方	ひとこと	出てくる所
成功か失敗かの型	`Result<i32, String>`		第0巻 UNIT 02
成功・失敗の値	`Ok(42)` `Err("だめ".to_string())`		第0巻 UNIT 02
失敗ならそこで返す	`let text = fs::read_to_string(path)?;`	呼んだ所へエラーを渡す	第0巻 UNIT 02
成功と失敗で分ける	`match r { Ok(v) => …, Err(e) => … }`		第0巻 UNIT 04
失敗なら止まる（試し書き）	`r.unwrap()` `r.expect("理由")`	本番では使わない	第0巻 UNIT 04
失敗なら決めた値にする	`r.unwrap_or(0)`		第0巻 UNIT 03
エラーを別の形にする	`r.map_err(|e| e.to_string())`		第2巻 UNIT 09
成功だけ Option にする	`r.ok()`		第2巻 UNIT 10
成功か調べる	`r.is_ok()` `r.is_err()`		第4巻 UNIT 06
main からエラーを返す	`fn main() -> Result<(), Box<dyn Error>>`		第3巻 UNIT 08
どんなエラーもまとめて返す	`fn main() -> anyhow::Result<()>`	anyhow を使う	第4巻 UNIT 05
エラーに説明を足す	`.context("設定を読めません")?`	anyhow	第4巻 UNIT 05
エラーを作ってすぐ返す	`bail!("題が空です");`	anyhow	第4巻 UNIT 05
自分のエラーの型を作る	`#[derive(Debug, thiserror::Error)] enum AppError { #[error("…")] Empty }`	thiserror	第4巻 UNIT 05
ほかのエラーから自動で変える	`#[error(transparent)] Io(#[from] std::io::Error)`	? で自動変換	第4巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
そこで止める（直せない間違い）	`panic!("ここには来ないはず");`		第5巻 UNIT 02
終了コードを付けて終わる	`std::process::exit(1);`		第4巻 UNIT 09
条件が合わなければ止める	`assert!(n > 0, "n は正の数");`		第0巻 UNIT 10
まだ書いていない所の印	`todo!()`	翻訳は通り、動くと止まる	第7巻 UNIT 07

関数・構造体・enum・トレイト

処理をまとめる関数、項目をまとめる構造体、種類を表す enum、共通の決まりを表すトレイトです。

やりたいこと	書き方	ひとこと	出てくる所
関数を作る	`fn add(a: i32, b: i32) -> i32 { a + b }`	最後の式が返る	第0巻 はじめに: 手を動かす準備
戻り値の型を書く	`-> i32`		第0巻 UNIT 02
途中で返す	`return Err(…);`		第0巻 UNIT 07
項目をまとめた型を作る	`struct Note { id: i64, title: String }`		第0巻 UNIT 08
型に関数（メソッド）を付ける	`impl Note { fn new(title: &str) -> Self { … } }`		第1巻 UNIT 08
自分を借りるメソッド	`fn len(&self) -> usize`	&mut self なら変えられる	第0巻 UNIT 07
残りの項目を、ほかの値と同じにする	`Note { title, ..old }`		第2巻 UNIT 07
種類を表す型を作る	`enum Status { Draft, Done }`		第2巻 UNIT 08
種類に値を持たせる	`enum Shape { Line { x1: f64, … }, Text(String) }`		第2巻 UNIT 08

やりたいこと	書き方	ひとこと	出てくる所
よく使う機能を自動で付ける	`#[derive(Debug, Clone, PartialEq)]`		第2巻 UNIT 07
初期値を作れるようにする	`#[derive(Default)]` `Note::default()`		第2巻 UNIT 10
並べられるようにする	`#[derive(PartialOrd, Ord, PartialEq, Eq)]`		第2巻 UNIT 10
表のキーにできるようにする	`#[derive(Hash, Eq, PartialEq)]`		第0巻 UNIT 08
共通の決まりを作る	`trait Summary { fn summary(&self) -> String; }`		第3巻 UNIT 08
型に決まりを付ける	`impl Summary for Note { … }`		第1巻 UNIT 08
型を変えられるようにする	`impl From<&str> for Title` `let t: Title = "a".into();`		第4巻 UNIT 05
確かめつつ型を変える	`impl TryFrom<String> for Title`	失敗はエラー	第4巻 UNIT 08
文字から作れるようにする	`impl FromStr for Status` `"done".parse::<Status>()`		第4巻 UNIT 01
どんな型でも受け取る（決まり付き）	`fn show<T: Display>(x: T)`		第4巻 UNIT 03
型を後で決める	`struct Wrapper<T> { value: T }`		第3巻 UNIT 05
決まりを満たす何かを返す	`fn items() -> impl Iterator<Item = i32>`		第1巻 UNIT 08
中身の型を後で決めて入れる	`Box<dyn Error>` `Vec<Box<dyn Shape>>`		第3巻 UNIT 08
名前の無い関数（クロージャ）	`let double = |x| x * 2;`		第0巻 UNIT 06
外の値を持っていくクロージャ	`move || println!("{name}")`	スレッドに渡す時	第3巻 UNIT 10
関数を引数で受け取る	`fn apply(f: impl Fn(i32) -> i32)`		第2巻 UNIT 10

やりたいこと	書き方	ひとこと	出てくる所
型に別名を付ける	`type Db = Arc<Mutex<Conn>>;`		第4巻 UNIT 01
その型の決まった値	`impl Note { const MAX: usize = 80; }`		第2巻 UNIT 02

モジュールと公開

コードをファイルやまとまりに分ける方法と、外から使えるかどうか（公開）の決め方です。

やりたいこと	書き方	ひとこと	出てくる所
ほかの場所の物を短い名前で使う	`use std::collections::HashMap;`		第0巻 UNIT 01
いくつかまとめて use する	`use std::{fs, io};`		第0巻 UNIT 01
全部まとめて use する	`use super::*;`	テストでよく使う	第0巻 UNIT 10
まとまり（モジュール）を作る	`mod auth;`	`src\auth.rs` を読む	第0巻 UNIT 10
外から使えるようにする	`pub fn` `pub struct`		第1巻 UNIT 06
このクレートの中だけ公開	`pub(crate) fn`		第4巻 UNIT 06
クレートのいちばん上から指す	`crate::auth::login`		第2巻 UNIT 10
1 つ上のモジュールを指す	`super::Note`		第0巻 UNIT 10
～の中の	`std::env::args`		第0巻 UNIT 01
テストの時だけ翻訳する	`#[cfg(test)] mod tests { … }`		第0巻 UNIT 10
機能が ON の時だけ翻訳する	`#[cfg(feature = "json")]`		第4巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
OS ごとに書き分ける	`#[cfg(windows)]` `#[cfg(unix)]`		第5巻 UNIT 06
使っていない警告を止める	`#[allow(dead_code)]`	本当に要らない時だけ	第4巻 UNIT 08
ファイル全体の説明を書く	`//! このファイルは…`		第4巻 UNIT 10
関数の説明を書く	`/// 題の文字数を数える`	cargo doc で読める	第1巻 UNIT 06
コメント	`// メモ` `/* メモ */`		第0巻 UNIT 01

所有権・借用・スマートポインタ

Rust の値には持ち主が 1 人だけいます。貸す（借用）・複製する・みんなで持つ、の書き方です。

やりたいこと	書き方	ひとこと	出てくる所
見せるだけ貸す	`&x`		第0巻 UNIT 02
変えてよい形で貸す	`&mut x`	同時には 1 つだけ	第0巻 UNIT 05
借りた先の値を使う・変える	`*count += 1;`		第2巻 UNIT 06
複製する	`x.clone()`	元も残る	第0巻 UNIT 09
借りた値から持ち物を作る	`s.to_owned()`		第2巻 UNIT 05
借りている期間に名前を付ける	`fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`		第3巻 UNIT 05
ずっと有効な参照	`&'static str`		第2巻 UNIT 08
値をヒープに置く	`Box::new(x)`	大きい値や、中身の型を後で決める時	第3巻 UNIT 08

やりたいこと	書き方	ひとこと	出てくる所
みんなで持つ（1 つのスレッド）	`Rc::new(x)` `Rc::clone(&a)`		第3巻 UNIT 09
みんなで持つ（スレッドをまたぐ）	`Arc::new(x)`		第3巻 UNIT 10
借りたまま中を変える	`RefCell::new(x)` `c.borrow_mut()`		第3巻 UNIT 09
スレッドをまたいで中を変える	`Arc::new(Mutex::new(0))` `*m.lock().unwrap() += 1`		第3巻 UNIT 10

読む人が多い時の鍵	`RwLock::new(x)`
	`.read()`
	`.write()`
			第3巻 UNIT 10

値を取り出して空にする	`std::mem::take(&mut v)`		＋ 合わせて覚える
2 つの値を入れかえる	`std::mem::swap(&mut a, &mut b)`		＋ 合わせて覚える

イテレーター（1 つずつ取り出して処理する）

並びの中身を 1 つずつ流して、変える・しぼる・集める道具です。`.collect()` などで最後に結果を作ります。

やりたいこと	書き方	ひとこと	出てくる所
借りて 1 つずつ	`v.iter()`		第0巻 UNIT 08
変えられる形で 1 つずつ	`v.iter_mut()`		第3巻 UNIT 03
持ち物として 1 つずつ（v は使えなくなる）	`v.into_iter()`		第2巻 UNIT 10
1 つずつ変える	`.map(|x| x * 2)`		第2巻 UNIT 08
条件に合う物だけ	`.filter(|n| !n.done)`		第2巻 UNIT 10
変えて、None を捨てる	`.filter_map(|s| s.parse().ok())`		第4巻 UNIT 09

やりたいこと	書き方	ひとこと	出てくる所
Vec などに集める	`.collect::<Vec<_>>()`		第0巻 UNIT 05
合計	`.sum::<i32>()`		第2巻 UNIT 10
数える	`.count()`		第0巻 UNIT 03
1 つにまとめる	`.fold(0, |acc, x| acc + x)`		第14巻 B. サンプル集
どれか 1 つでも合うか	`.any(|x| x > 10)`		第2巻 UNIT 10
全部が合うか	`.all(|x| x > 0)`		第2巻 UNIT 10
最初に合う物	`.find(|n| n.id == id)`		第0巻 UNIT 05
何番目に合うか	`.position(|x| x == 3)`		第3巻 UNIT 07
いちばん大きい・小さい	`.max()` `.min()`		第2巻 UNIT 02
項目でいちばんを選ぶ	`.max_by_key(|n| n.updated_at)`		第14巻 B. サンプル集
何番目かを付ける	`.enumerate()`		第0巻 UNIT 03
2 つを組にする	`a.iter().zip(b.iter())`		第7巻 この巻で身に付けること
最初の n 個	`.take(5)`		第0巻 UNIT 05
最初の n 個を飛ばす	`.skip(5)`	ページ送りに	第0巻 UNIT 03
後ろにつなぐ	`a.iter().chain(b.iter())`		第5巻 UNIT 09

やりたいこと	書き方	ひとこと	出てくる所
逆の順	`.rev()`		第2巻 UNIT 04
中の並びを平らにする	`.flat_map(|s| s.chars())`		第14巻 B. サンプル集
借りた値を複製して流す	`.copied()` `.cloned()`		第2巻 UNIT 10
次を先に見る	`let mut it = v.iter().peekable();` `it.peek()`		第7巻 この巻で身に付けること
条件の間だけ取る	`.take_while(|x| x < 10)`		＋ 合わせて覚える
最後の 1 つ	`.last()`		第4巻 UNIT 06
1 つずつ処理するだけ	`.for_each(|x| println!("{x}"))`		第2巻 UNIT 10

ファイルとフォルダ

`use std::fs;`・`use std::path::Path;` を使います。失敗するかもしれないので、多くは `Result` を返します（`?` で受けます）。

やりたいこと	書き方	ひとこと	出てくる所
ファイルを全部、文字として読む	`let text = fs::read_to_string("memo.txt")?;`		第0巻 UNIT 02
ファイルを全部、バイトで読む	`let bytes = fs::read("photo.png")?;`		第12巻 UNIT 01
ファイルに書く（上書き）	`fs::write("memo.txt", "中身")?;`	前の中身は消える	第0巻 UNIT 02
ファイルに書き足す	`OpenOptions::new().append(true).create(true).open("log.txt")?`		第0巻 UNIT 07
新しいファイルを作って書く	`let mut f = File::create("out.txt")?; writeln!(f, "1 行")?;`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
ファイルを開いて読む	`let f = File::open("in.txt")?;`		第9巻 UNIT 09
1 行ずつ読む	`for line in BufReader::new(f).lines() { let line = line?; }`	大きなファイルでも少しずつ	第0巻 UNIT 06
まとめて速く書く	`let mut w = BufWriter::new(f);`		＋ 合わせて覚える
フォルダを作る（途中も）	`fs::create_dir_all("data/notes")?;`	もうあってもよい	第0巻 UNIT 02
フォルダの中を一覧する	`for e in fs::read_dir(".")? { let e = e?; e.path(); }`	順番は決まっていない	第0巻 UNIT 02
ファイルを消す	`fs::remove_file("tmp.txt")?;`		第0巻 UNIT 07
フォルダを中身ごと消す	`fs::remove_dir_all("tmp")?;`	戻せない	第14巻 B. サンプル集
名前を変える・動かす	`fs::rename("a.txt", "b.txt")?;`		第14巻 B. サンプル集
コピーする	`fs::copy("a.txt", "b.txt")?;`		第14巻 B. サンプル集
あるか調べる	`Path::new("memo.txt").exists()`		第0巻 UNIT 02
フォルダか・ファイルか	`p.is_dir()` `p.is_file()`		第0巻 UNIT 02
大きさ・日時を調べる	`fs::metadata(p)?.len()` `.modified()?`		第14巻 B. サンプル集
場所をつなぐ	`Path::new("memo").join("today.txt")`	OS に合った区切り	第0巻 UNIT 02
場所の部品を取り出す	`p.file_name()` `p.extension()` `p.parent()`		第0巻 UNIT 02
拡張子を除いた名前	`p.file_stem()`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
拡張子を変える	`p.with_extension("bak")`		＋ 合わせて覚える
場所を表示する	`println!("{}", p.display());`		第0巻 UNIT 02
`C:\` から始まる場所にする	`fs::canonicalize(p)?`		＋ 合わせて覚える
今いるフォルダ	`env::current_dir()?`		第0巻 UNIT 02
今いるフォルダを変える	`env::set_current_dir("work")?;`		＋ 合わせて覚える
持てる場所の型	`PathBuf::from("data")`	変えたり足したりできる	第9巻 UNIT 04
一時フォルダ	`env::temp_dir()`		第11巻 UNIT 02
無いファイルの時だけ分ける	`Err(e) if e.kind() == io::ErrorKind::NotFound => …`		第4巻 UNIT 04
テスト用の一時フォルダ	`let dir = tempfile::tempdir()?;`	終わると消える	第9巻 UNIT 05

入力・引数・環境変数・ほかのプログラム

プログラムの外から値を受け取る方法と、ほかのプログラムを動かす方法です。

やりたいこと	書き方	ひとこと	出てくる所
キーボードから 1 行読む	`io::stdin().read_line(&mut line)?;`	最後に改行が付く	第0巻 UNIT 06
キーボードから全部読む	`io::stdin().lines()`	1 行ずつ	第2巻 UNIT 10
引数を受け取る	`let args: Vec<String> = env::args().collect();`	0 番目はプログラムの名前	第0巻 UNIT 01
引数を 1 つずつ見る	`let mut args = env::args().skip(1); args.next()`		第0巻 UNIT 06

やりたいこと	書き方	ひとこと	出てくる所
環境変数を読む	`env::var("DATABASE_URL")?`	無いとエラー	第0巻 UNIT 03
環境変数が無ければ決めた値	`env::var("PORT").unwrap_or_else(|_| "3000".into())`		第4巻 UNIT 05
.env のファイルを読む	`dotenvy::dotenv().ok();`	開発の時の設定	第5巻 UNIT 01
OS の種類を知る	`env::consts::OS`	"windows" など	第0巻 UNIT 01
ほかのプログラムを動かす	`Command::new("git").arg("--version").output()?`		＋ 合わせて覚える
動かして終わりを待つ	`Command::new("cargo").args(["fmt", "--check"]).status()?`	.success() で成功か	＋ 合わせて覚える
出力を文字にする	`String::from_utf8_lossy(&out.stdout)`		第5巻 UNIT 08
すぐ画面に出す	`io::stdout().flush()?;`	print! の後に	第2巻 UNIT 10

時間

時間を測る・待つ・日時を扱う方法です。

やりたいこと	書き方	ひとこと	出てくる所
処理の時間を測る	`let t = Instant::now(); … t.elapsed()`	{:?} で 1.2ms のように出る	第0巻 UNIT 04
時間の長さを作る	`Duration::from_secs(5)` `Duration::from_millis(200)`		第5巻 UNIT 02
待つ（スレッド）	`thread::sleep(Duration::from_millis(100));`		第3巻 UNIT 10
待つ（async の中）	`tokio::time::sleep(Duration::from_secs(1)).await;`	スレッドを止めない	第5巻 UNIT 01
決めた時間で打ち切る	`tokio::time::timeout(Duration::from_secs(5), fut).await`		第5巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
決まった間かくでくり返す	`let mut tick = tokio::time::interval(Duration::from_secs(60)); tick.tick().await;`		第5巻 UNIT 02
今の日時（1970 年からの秒）	`SystemTime::now().duration_since(UNIX_EPOCH)?.as_secs()`		第4巻 UNIT 02
秒・ミリ秒にする	`d.as_secs()` `d.as_millis()`		第4巻 UNIT 02
今の日時（time）	`OffsetDateTime::now_utc()`	time を使う	＋ 合わせて覚える
日時を文字にする（chrono）	`now.format("%Y-%m-%d %H:%M").to_string()`	%Y 年 %m 月 %d 日	＋ 合わせて覚える

スレッドと async（同時に動かす）

いくつもの仕事を同時に進める方法です。Web のサーバーは `tokio` の async で動きます。

やりたいこと	書き方	ひとこと	出てくる所
スレッドを作る	`let h = thread::spawn(move || { … });`		第3巻 UNIT 10
スレッドの終わりを待つ	`h.join().unwrap()`		第0巻 UNIT 02
スレッドどうしで値を送る	`let (tx, rx) = mpsc::channel(); tx.send(x)?; rx.recv()?`		第5巻 UNIT 04
みんなで数を変える	`let n = Arc::new(Mutex::new(0));`		第3巻 UNIT 10
止まらずに数える	`AtomicUsize::new(0)` `.fetch_add(1, Ordering::Relaxed)`		第11巻 UNIT 08
async の関数を作る	`async fn load(id: i64) -> Result<Note>`		第1巻 UNIT 08
終わるまで待つ	`let note = load(1).await?;`		第5巻 UNIT 01
main を async にする	`#[tokio::main] async fn main()`		第5巻 UNIT 01

やりたいこと	書き方	ひとこと	出てくる所
1 本のスレッドで動かす	`#[tokio::main(flavor = "current_thread")]`		第5巻 UNIT 05
裏で仕事を走らせる	`let h = tokio::spawn(async move { … });`		第5巻 UNIT 02
いくつかを同時に待つ	`let (a, b) = tokio::join!(f1, f2);`		第5巻 UNIT 01
先に終わった方を使う	`tokio::select! { r = f1 => …, _ = f2 => … }`		第5巻 UNIT 02
重い計算を別のスレッドで	`tokio::task::spawn_blocking(move || hash(&pw)).await?`	パスワードのハッシュなど	第5巻 UNIT 02
たくさんの仕事をまとめて待つ	`let mut set = JoinSet::new(); set.spawn(…); set.join_next().await`		第5巻 UNIT 03
止める合図	`CancellationToken::new() .cancel()`	tokio-util	第5巻 UNIT 06
async で値を送る	`let (tx, mut rx) = tokio::sync::mpsc::channel(100);`		第5巻 UNIT 04
async の鍵	`tokio::sync::Mutex::new(x) .lock().await`	.await の間も持つ時	第3巻 UNIT 10

Web の画面と URL（Topcoat）

この本で使う Topcoat（Rust の Web アプリの部品）の、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所
URL に答えるページを作る	`#[page] async fn home(cx: &Cx) -> Result<impl View>`		第1巻 UNIT 08
フォームの送信（POST）に答える	`#[page(POST)]`		第5巻 UNIT 08
JSON や文字を返す	`#[route(GET)]`		第6巻 UNIT 03
全部のページの外枠を作る	`#[layout]`		第1巻 UNIT 08

やりたいこと	書き方	ひとこと	出てくる所
HTML を書く	`view! { <h1>"こんにちは"</h1> }`	文字は " で囲む	第1巻 UNIT 08
値を HTML の中に入れる	`view! { <p>(note.title)</p> }`	( ) の中は式	第1巻 UNIT 08
くり返して表示する	`for note in notes { <li>(note.title)</li> }`	#[key(…)] を付ける	第6巻 UNIT 05
条件で表示を変える	`if done { "済" } else { "未" }`		第0巻 UNIT 02
クラスを条件で付ける	`class=(class!("note", "done" if note.done))`		第6巻 UNIT 03
URL を作る	`href!(show, Id(note.id))`		第5巻 UNIT 08
部品（コンポーネント）を作る	`#[component] fn panel(title: &str, #[default] child: Child<'_>)`		第6巻 UNIT 05
CSS などのファイルを使う	`asset!("styles.css")`		第1巻 UNIT 08
クエリ（?q=…）を読む	`#[query_params(error = bad_request)] struct Q { q: Option<String> }`		第6巻 UNIT 04
1 回の要求の中で結果を覚える	`#[memoize(as_ref)] async fn load_user(cx: &Cx, id: u32)`		第6巻 UNIT 07
ブラウザーの中で動く状態	`let count = signal(cx, || 1usize);`		第6巻 UNIT 09
ブラウザーでも動く式	`$(count.get())`		第6巻 UNIT 09
押した時の処理（ブラウザーの中）	`@click=$(|_e| count.increment())`		第6巻 UNIT 09
サーバーの関数をブラウザーから呼ぶ	`#[procedure("/api/double")]`		第6巻 UNIT 09
ルーターを作って起動する	`let router = Router::builder().discover().build(); topcoat::start(router).await`		第6巻 UNIT 02

やりたいこと	書き方	ひとこと	出てくる所
入力欄のある送信フォーム	`<form method="post"><input name="title"><button>"保存"</button></form>`		第6巻 UNIT 06
送ったあと別のページへ移す	`Err(see_other(href!(show, Id(note.id)).resolve(cx)).into())`	303 で移す。PRG（二重送信を防ぐ）	第5巻 UNIT 08
状態コードを返す	`StatusCode::NOT_FOUND`	404	第5巻 UNIT 09
ログインしている人を調べる	`current_user(cx).await?`	RustNote で作る関数	第8巻 UNIT 10
クッキーを読む・付ける	`cookies(cx)` `Cookie::new("名前", "値")`		第6巻 UNIT 10
ほかのサーバーに HTTP で送る	`reqwest::get(url).await?.text().await?`	reqwest を使う	第3巻 UNIT 06
JSON を送って受け取る	`client.post(url).json(&body).send().await?.json::<T>().await?`		第5巻 UNIT 08

データベースと SQL

データベース（Turso・SQLite）に、Rust からつないで SQL を送る方法と、よく使う SQL です。

やりたいこと	書き方	ひとこと	出てくる所
SQLite のファイルにつなぐ	`let db = Builder::new_local("app.db").build().await?; let conn = db.connect()?;`		第5巻 UNIT 10
メモリの中のデータベース	`Builder::new_local(":memory:")`	終わると消える	第5巻 UNIT 10
変える SQL を送る	`conn.execute("DELETE FROM notes WHERE id = ?1", [id]).await?`	変わった行の数が返る	第5巻 UNIT 10
読む SQL を送る	`let mut rows = conn.query("SELECT id, title FROM notes", ()).await?;`		第5巻 UNIT 01

やりたいこと	書き方	ひとこと	出てくる所
行を 1 つずつ読む	`while let Some(row) = rows.next().await? { let id: i64 = row.get(0)?; }`		第5巻 UNIT 01
値を安全に渡す	`WHERE id = ?1 AND user_id = ?2`	文字をつながない	第5巻 UNIT 01
名前で値を渡す	`named_params! { ":owner": 1 }`		第7巻 UNIT 07
まとめて成功・失敗にする	`let tx = conn.transaction().await?; … tx.commit().await?;`		第7巻 UNIT 04
最後に入れた行の番号	`conn.last_insert_rowid()`		第7巻 UNIT 07
表を作る	`CREATE TABLE notes (id INTEGER PRIMARY KEY, title TEXT NOT NULL)`		第0巻 UNIT 07
無ければ作る	`CREATE TABLE IF NOT EXISTS …`		第5巻 UNIT 10
値の決まりを付ける	`CHECK (length(title) BETWEEN 1 AND 80)`		第0巻 UNIT 07
重なりを禁止する	`email TEXT NOT NULL UNIQUE`		第0巻 UNIT 07
ほかの表の行を指す	`user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE`		第0巻 UNIT 07
初めの値を決める	`created_at TEXT NOT NULL DEFAULT (datetime('now'))`		第0巻 UNIT 07
行を入れる	`INSERT INTO notes (title) VALUES (?1)`		第0巻 UNIT 07
あれば何もしない	`INSERT OR IGNORE INTO … ON CONFLICT DO NOTHING`		第5巻 UNIT 10
あれば書きかえる	`ON CONFLICT(email) DO UPDATE SET name = excluded.name`		第8巻 UNIT 03
読む	`SELECT id, title FROM notes WHERE done = 0`		第0巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
全部の列	`SELECT * FROM notes`	試しに見る時だけ	第7巻 UNIT 01
並べる	`ORDER BY updated_at DESC`	DESC は大きい順	第5巻 UNIT 10
数をしぼる・飛ばす	`LIMIT 20 OFFSET 40`	ページ送り	第5巻 UNIT 03
文字の一部で探す	`WHERE title LIKE '%買い物%'`	% は何でも	第7巻 UNIT 02
% や _ そのものを探す	`LIKE ?1 ESCAPE '\'`		第7巻 UNIT 02
数える	`SELECT COUNT(*) FROM notes`		第5巻 UNIT 10
グループごとに数える	`SELECT status, COUNT(*) FROM jobs GROUP BY status`		第5巻 UNIT 10
重なりを消す	`SELECT DISTINCT user_id FROM notes`		第10巻 UNIT 01
2 つの表をつなぐ	`SELECT n.title, u.email FROM notes n JOIN users u ON u.id = n.user_id`		第7巻 UNIT 02
相手が無くても残してつなぐ	`LEFT JOIN`		第7巻 UNIT 02
書きかえる	`UPDATE notes SET done = 1 WHERE id = ?1`	WHERE を忘れると全部変わる	第5巻 UNIT 10
消す	`DELETE FROM notes WHERE id = ?1`		第7巻 この巻で身に付けること
変えた行を返す	`… RETURNING id`		第5巻 UNIT 10
空なら別の値	`COALESCE(body, '')`		第7巻 UNIT 09
文字をつなぐ	`'user' || n || '@example.com'`		第7巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
今の日時	`datetime('now')` `unixepoch()`		第7巻 UNIT 01
速く探すための索引	`CREATE INDEX idx_notes_user ON notes(user_id)`		第7巻 UNIT 01
索引が使われるか見る	`EXPLAIN QUERY PLAN SELECT …`		第7巻 この巻で身に付けること
表の形を変える	`ALTER TABLE notes ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0`		第7巻 UNIT 06
SQLite の設定	`PRAGMA foreign_keys = ON` `PRAGMA journal_mode = WAL`		第5巻 UNIT 06
写しを書き出す	`VACUUM INTO 'backup.db'`		第7巻 UNIT 10
SQL のコメント	`-- メモ`		第7巻 この巻で身に付けること

JSON（serde）

Rust の値と JSON を行き来させる方法です。`serde`（`features = ["derive"]`）と `serde_json` を使います。

やりたいこと	書き方	ひとこと	出てくる所
型を JSON にできるようにする	`#[derive(Serialize, Deserialize)]`		第4巻 UNIT 07
JSON の文字にする	`serde_json::to_string(&note)?`		第4巻 UNIT 07
見やすい JSON の文字にする	`serde_json::to_string_pretty(&note)?`		第4巻 UNIT 08
JSON の文字から読む	`let n: Note = serde_json::from_str(&text)?;`		第4巻 UNIT 08
その場で JSON を作る	`json!({ "ok": true, "id": id })`		第14巻 B. サンプル集

やりたいこと	書き方	ひとこと	出てくる所
形の決まっていない JSON	`serde_json::Value`	v["title"] で読む	第7巻 この巻で身に付けること
名前の書き方を変える	`#[serde(rename_all = "camelCase")]`	width_mm → widthMm	第4巻 UNIT 08
1 つの項目の名前を変える	`#[serde(rename = "type")]`		＋ 合わせて覚える
無い項目は初期値にする	`#[serde(default)]`		第4巻 UNIT 08
空なら書かない	`#[serde(skip_serializing_if = "Option::is_none")]`		第4巻 UNIT 08
知らない項目をエラーにする	`#[serde(deny_unknown_fields)]`		第4巻 UNIT 08
読む時に確かめて変える	`#[serde(try_from = "String")]`		第4巻 UNIT 08
enum の種類を項目で表す	`#[serde(tag = "kind")]`		＋ 合わせて覚える

ログ（tracing）

動いている様子を記録する方法です。`tracing` と `tracing-subscriber` を使います。

やりたいこと	書き方	ひとこと	出てくる所
ログを出す準備をする	`tracing_subscriber::fmt().init();`	main の最初に	第5巻 UNIT 09
ふつうのお知らせ	`info!("起動しました");`		第5巻 UNIT 09
気になること	`warn!("遅い: {ms}ms");`		第5巻 UNIT 09
エラー	`error!(error = ?e, "保存できません");`		第5巻 UNIT 09
くわしい調べ物用	`debug!("…")` `trace!("…")`		第5巻 UNIT 09

やりたいこと	書き方	ひとこと	出てくる所
欄を付けて記録する	`info!(user_id = %id, "ログイン");`	% は {} の形	第5巻 UNIT 09
Debug の形で記録する	`warn!(err = ?e);`	? は {:?} の形	第5巻 UNIT 09
関数の呼び出しを記録する	`#[instrument(skip(db))]`	秘密は skip	第5巻 UNIT 09
どこまで細かく出すか決める	`$env:RUST_LOG = "info,rustnote=debug"`		第5巻 UNIT 09

テスト

書いたコードが正しく動くかを、自動で確かめる方法です。`cargo test` で動きます。

やりたいこと	書き方	ひとこと	出てくる所
テストを書く	`#[test] fn title_is_trimmed() { … }`		第0巻 UNIT 10
テストをまとめる	`#[cfg(test)] mod tests { use super::*; … }`		第0巻 UNIT 10
等しいか確かめる	`assert_eq!(add(2, 3), 5);`	ちがうと両方の値を表示	第0巻 UNIT 10
ちがうか確かめる	`assert_ne!(a, b);`		第8巻 UNIT 10
true か確かめる	`assert!(v.is_empty(), "空のはず");`		第0巻 UNIT 10
形が合うか確かめる	`assert!(matches!(r, Err(AppError::Empty)));`		第2巻 UNIT 08
止まるのが正しいと確かめる	`#[should_panic(expected = "Empty")]`		第4巻 UNIT 10
ふだんは飛ばす	`#[ignore]`	cargo test -- --ignored で動く	第11巻 UNIT 06
async のテスト	`#[tokio::test] async fn …`		第5巻 UNIT 01

やりたいこと	書き方	ひとこと	出てくる所
テストで Result を返す	`#[test] fn t() -> Result<(), Box<dyn Error>> { …; Ok(()) }`	? が使える	第0巻 UNIT 10
外から使うテスト（tests フォルダに置く）	`tests\http.rs`	cargo test --test http で動かす	第8巻 UNIT 01
説明の中の例をテストにする	`/// ``` /// assert_eq!(…);`	cargo test --doc	第4巻 UNIT 10
失敗した場所を呼んだ所にする	`#[track_caller]`		第11巻 UNIT 03
速さを比べるための値	`std::hint::black_box(x)`	最適化で消されない	第0巻 UNIT 04

安全（パスワード・合言葉・HTML）

ログインの仕組みで使う、パスワードと合言葉（トークン）の扱い方です。

やりたいこと	書き方	ひとこと	出てくる所
パスワードをハッシュにする	`Argon2::default().hash_password(pw.as_bytes(), &salt)?.to_string()`	元に戻せない形	第5巻 UNIT 05
パスワードを確かめる	`Argon2::default().verify_password(pw.as_bytes(), &parsed).is_ok()`		第5巻 UNIT 07
塩（salt）を作る	`SaltString::generate(&mut OsRng)`		第8巻 UNIT 02
予想できない合言葉を作る	`let mut b = [0u8; 32]; getrandom::fill(&mut b)?;`		＋ 合わせて覚える
16 進数の文字にする	`hex::encode(b)`		＋ 合わせて覚える
指紋（SHA-256）を作る	`Sha256::digest(token.as_bytes())`		第8巻 UNIT 09
同じかを、時間の差を出さずに比べる	`a.ct_eq(&b)`	subtle を使う	＋ 合わせて覚える
Base64 にする	`STANDARD.encode(bytes)`		第8巻 UNIT 09

やりたいこと	書き方	ひとこと	出てくる所

HTML の特別な文字を変える	`< → <`
	`> → >`
	`& → &`
		view! は自動で変える	＋ 合わせて覚える

クッキーを安全にする	`HttpOnly; Secure; SameSite=Lax`	JavaScript から読めない・HTTPS だけ	第10巻 UNIT 09

HTML と CSS

画面の中身（HTML）と見た目（CSS）の、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所
ページの最初の宣言	`<!DOCTYPE html>`		第1巻 UNIT 08
言葉と文字の符号	`<html lang="ja">` `<meta charset="utf-8">`		第1巻 UNIT 08
スマホの幅に合わせる	`<meta name="viewport" content="width=device-width, initial-scale=1">`		第1巻 UNIT 08
CSS のファイルをつなぐ	`<link rel="stylesheet" href="style.css">`		第1巻 UNIT 08
リンク	`<a href="/notes">一覧</a>`		第1巻 UNIT 08
見出し	`<h1>…</h1>` `<h2>…</h2>`		第6巻 UNIT 05
送信フォーム	`<form method="post" action="/notes">`		第6巻 UNIT 06
入力欄	`<input name="title" type="text" required maxlength="80">`		第6巻 UNIT 05
入力欄の名前の説明	`<label for="title">題</label>`		第9巻 UNIT 02
何行も書ける欄	`<textarea name="body" rows="6"></textarea>`		第13巻 UNIT 07

やりたいこと	書き方	ひとこと	出てくる所
選ぶ欄	`<select name="kind"><option value="a">A</option></select>`		＋ 合わせて覚える
チェック	`<input type="checkbox" checked>`		第6巻 UNIT 05
ボタン	`<button type="submit">保存</button>`		第6巻 UNIT 06
箇条書き	`<ul><li>…</li></ul>`		第6巻 UNIT 05
開いたり閉じたりする部分	`<details><summary>見出し</summary>…</details>`		第13巻 UNIT 08
お知らせを読み上げてもらう	`role="status" aria-live="polite"`		第10巻 UNIT 09
読み上げ用の名前	`aria-label="閉じる"`		第6巻 UNIT 03
クラスで見た目を付ける	`.note { color: #334155; }`		第13巻 UNIT 09
マウスを乗せた時	`button:hover { … }`		第13巻 UNIT 09
色を変数にする	`:root { --main: #2563eb; }` `color: var(--main);`		第9巻 UNIT 09
横に並べる	`display: flex; gap: 8px;`		第13巻 UNIT 09
格子に並べる	`display: grid; grid-template-columns: 1fr 2fr;`		＋ 合わせて覚える
画面が狭い時だけ	`@media (max-width: 640px) { … }`		＋ 合わせて覚える
文字の大きさ	`font-size: 1rem;`	rem はページの基本の何倍か	第6巻 UNIT 08
余白	`margin: 0 auto; padding: 8px 12px;`		第6巻 UNIT 08
枠線と角の丸み	`border: 1px solid #cbd5e1; border-radius: 6px;`		第13巻 UNIT 09

やりたいこと	書き方	ひとこと	出てくる所
幅の数え方をそろえる	`* { box-sizing: border-box; }`		第13巻 UNIT 09

正規表現（文字の形で探す）

「数字 6 けた」のような文字の形を、記号で表す書き方です。Rust では `regex` を使います。

やりたいこと	書き方	ひとこと	出てくる所
形に合うか調べる	`Regex::new(r"^\d{6}$")?.is_match(s)`		第1巻 UNIT 01
数字 1 つ	`\d`		第11巻 UNIT 05
英数字か _ の 1 文字	`\w`		＋ 合わせて覚える
空白 1 つ	`\s`		第8巻 UNIT 08
何でも 1 文字	`.`		第12巻 UNIT 04

0 回以上・1 回以上・あってもなくても	`*`
	`+`
	`?`
			第8巻 UNIT 08

ちょうど n 回・n〜m 回	`{6}` `{8,12}`		第9巻 UNIT 09
頭・終わり	`^` `$`		＋ 合わせて覚える
どれか 1 文字・それ以外の 1 文字	`[0-9a-z]` `[^;]`		第9巻 UNIT 09
まとまりと取り出し	`(\d+)`	caps[1] で取り出す	第9巻 UNIT 09
どちらか	`api/|admin/`		＋ 合わせて覚える
記号そのもの	`\.` `\/`		第11巻 UNIT 05

やりたいこと	書き方	ひとこと	出てくる所
合う所を取り出す	`re.captures(s)` `re.find_iter(s)`		＋ 合わせて覚える
合う所を置きかえる	`re.replace_all(s, "-")`		＋ 合わせて覚える

設定ファイル（Cargo.toml・fly.toml・Dockerfile・GitHub Actions）

プロジェクトや公開の設定ファイルの、よく使う書き方です。

やりたいこと	書き方	ひとこと	出てくる所

パッケージの名前と版	`[package]`
	`name = "practice"`
	`edition = "2024"`
			第0巻 はじめに: 手を動かす準備

使う部品	`[dependencies]` `serde = { version = "1", features = ["derive"] }`		第0巻 はじめに: 手を動かす準備
版をぴったり決める	`tokio = "=1.54.1"`	= を付ける	第1巻 UNIT 08
テストの時だけの部品	`[dev-dependencies]`		第9巻 UNIT 05
ON・OFF できる機能	`[features]` `default = ["markdown"]`		第4巻 UNIT 07
本番用の作り方	`[profile.release]` `overflow-checks = true`		第12巻 UNIT 01
いくつかのパッケージをまとめる	`[workspace]` `members = ["app", "domain"]`		第4巻 UNIT 06
いちばん古く使える Rust の版	`rust-version = "1.99"`		第2巻 UNIT 10
Fly.io のアプリの名前と場所	`app = "…"` `primary_region = "nrt"`		第12巻 UNIT 04
Fly.io に渡す環境変数	`[env]` `PORT = "8080"`	秘密は fly secrets	第12巻 UNIT 04

やりたいこと	書き方	ひとこと	出てくる所
Fly.io の待ち受けの設定	`[http_service]` `internal_port = 8080`		第12巻 UNIT 04
元気さの確かめ	`[[http_service.checks]]` `path = "/health"`		第9巻 UNIT 10
公開の前に動かすコマンド	`[deploy]` `release_command = "…"`		第12巻 UNIT 07
Dockerfile の始まり	`FROM rust:1.99.1-trixie AS builder`		第12巻 UNIT 01
作業するフォルダ	`WORKDIR /app`		第12巻 UNIT 01
コピーする	`COPY . .` `COPY --from=builder /app/target/release/app /app/app`		第12巻 UNIT 01
組み立てる途中のコマンド	`RUN cargo build --release --locked`		第12巻 UNIT 01
動かす利用者	`USER app`	root で動かさない	第12巻 UNIT 02
動かすプログラム	`CMD ["/app/app"]`		第12巻 UNIT 02

GitHub Actions の動く時	`on:`
	`push:`
	`branches: [main]`
			第11巻 UNIT 10

GitHub Actions の仕事と手順	`jobs:`
	`runs-on: ubuntu-26.04`
	`steps:`
			第11巻 UNIT 10

手順でコマンドを動かす	`- run: cargo test --locked`		第1巻 UNIT 03
決まった手順を使う	`- uses: actions/checkout@v7`		第11巻 UNIT 10
毎日決まった時刻に動かす	`schedule:` `- cron: "0 0 * * *"`	UTC の 0 時	第11巻 UNIT 10
VS Code の保存の時に整える	`"editor.formatOnSave": true`		第1巻 UNIT 03

ブラウザーで確かめる（Playwright・JavaScript）

本物のブラウザーを動かして、画面を確かめるテストの書き方です。

やりたいこと	書き方	ひとこと	出てくる所
テストを 1 つ書く	`test("ログインできる", async ({ page }) => { … });`		第11巻 UNIT 05
ページを開く	`await page.goto("/login");`		第11巻 UNIT 05
ラベルで入力欄を探して入れる	`await page.getByLabel("メールアドレス").fill("a@example.com");`		第11巻 UNIT 05
ボタンを押す	`await page.getByRole("button", { name: "ログイン" }).click();`		第11巻 UNIT 05
URL を確かめる	`await expect(page).toHaveURL(/\/notes$/);`		第11巻 UNIT 05
文字が出ているか確かめる	`await expect(page.getByText("保存しました")).toBeVisible();`		第11巻 UNIT 05
画面の写しを撮る	`await page.screenshot({ path: "a.png" });`		＋ 合わせて覚える
テストを動かす	`npx playwright test`		第11巻 UNIT 05
文字の中に値を入れる（JS）	`` `${name} さん` ``		第8巻 UNIT 08
JSON にする・読む（JS）	`JSON.stringify(obj)` `JSON.parse(text)`		第8巻 UNIT 08